SV8_Week 1

by Reem Hamada

Be aware: This document does not cover all possible correct answers. If your answer differs, it does not necessarily mean it is incorrect.

Q1 *

FSM (Finite State Machine) coverage includes:

- ○ a. State coverage – check if each state is reached
- ○ b. Transition coverage – check if each valid state transition occurs
- ○ c. Both a & b
- ○ d. None of the above

Answer:

c. Both a & b

Q2 *

Once you reach 100% code coverage that indicates correctness of the design.

- ○ True
- ○ False

Answer:

False

Justification:

100% code coverage indicates that every line, branch, and statement in the RTL has been executed at least once during simulation.

Code coverage reflects the extent to which different portions of the design code have been stimulated, but it does not guarantee that the DUT is functioning according to its specifications.

Q3) *

A 5-bit signal is monitored for toggle coverage.
How many unique bit transitions must occur for 100% toggle coverage?

- ○ 5
- ○ 25
- ○ 32
- ○ 10
- ○ Depends on simulation time

Answer:

10

Justification:

For 5 bits, each bit must undergo two transitions: 0 → 1 and 1 → 0. That results in 2 transitions per bit. For full coverage, the total number of transitions required is

5 * 2 = 10.

Q4) Hint: take care of the used data types *

```
module tb4;
  bit   [3:0] a;
  logic [3:0] A;
  logic [3:0] b;
  logic [3:0] c;

  initial begin
    a = 4'b1xz0;
    b = 4'b1xz0;
    A = 4'b1xz0;

    c = a & b;

    $display("c = %b", c);
    $display("(a === b) equals %0d", (a === b));
    $display("(A === b) equals %0d", (A === b));
    $display("(a == A) equals %0d",  (a == A));
  end
endmodule
```

None is correct

c = 1xz0
(a === b) equals 0
(A === b) equals 1
(a == A) equals 0

Option 5

c = 1000
(a === b) equals 0
(A === b) equals 1
(a == A) equals x

Option 4

c = 1000
(a === b) equals 1
(A === b) equals 1
(a == A) equals 0

Option 1

c = 1xz0
(a === b) equals 0
(A === b) equals 1
(a == A) equals 1

Option 2

Answer:

Option 4

Justification:

a = 1000; b = 1xz0; (zero ANDed with anything is zero)

c = 1000;

Q5 * 10 points

What is the disadvantage of using variables of type 'bit'?

- ◯ They take more memory compared to logic
- ◯ They cannot be assigned inside procedural blocks
- ◯ They cannot represent unknown (X) or high-impedance (Z) states
- ◯ They cannot be used in functions

Answer:

They cannot represent unknown (X) or high impedance (Z) states.

Note:

This is exactly why we avoid using 2-state data types for output signals in testbenches, because if the output enters an unknown (X) or high-impedance (Z) state, it will be represented as 0, making these conditions invisible to us.

Q6) Calculate the coverage percentage for "branch coverage" and "toggle coverage" and choose the correct. *

```
module mux4to1 (
    input  [3:0] d,
    input  [1:0] sel,
    output reg   y
);

always @(*) begin
    case (sel)
        2'b00:   y = d[0];
        2'b01:   y = d[1];
        2'b10:   y = d[2];
        2'b11:   y = d[3];
        default: y = 1'b0;
    endcase
end

endmodule
```

```
module tb_mux4to1;

  bit [3:0] d;
  bit [1:0] sel;
  logic     y;

  mux4to1 dut (.*);

  initial begin
    d   = 4'b1010;
    sel = 2'b00;

    #10 sel = 3'b10;
    #10 sel = 3'b01;

    #20 $stop;
  end

endmodule
```

	60%	100%	28.57%	42.85%	66.66%
Branch Covera...	○	○	○	○	○
Toggle Coverage	○	○	○	○	○

Answer:

Branch Coverage = 60%
Toggle Coverage = 28.57%

Justification:

Branch Coverage Calc: total of **5 lines** (line #9 to line #13), only the first **3 lines** got exercised by the testbench.

Therefore, coverage: number of line hits / total number of branch lines

3/5 *100 = 60%

Toggle Coverage Calc: total of **7 bits** (4 bits of "d", 2 bits of "sel," and 1 bit of "y") need to be fully toggled from 1-> 0 and from 0 ->1

So total of 7*2 = 14 transitions

d => 0/8 transitions occurred.
sel => 3/4 transitions occurred.
y => 1/2 transitions occurred.

Therefore, coverage: number of transitions that occurred / total number of possible transitions

4/14 *100 = 28.57%

Q7 * 10 points

A Verilog module can be tested by a SystemVerilog testbench.

- ◯ Wrong, design need to be re-written in SystemVerilog
- ◯ Correct

Answer:

Correct

Justification:

SystemVerilog is built on Verilog and, therefore, compatible with Verilog. (Back Compatibility Feature)

Q8) Upload screenshots ONLY * 10 points

How to use Srandom to randomize "a" with positive, even numbers from 7 to 21 inclusively, and divisible by 2.

No use of **Surandrange** is allowed.

Usage of if statement and expressions are allowed. Feel free to run it on questasim for trials.

```
1   module tb_try;
2
3     integer a;
4
5     initial begin
6       repeat (1000) begin
7
8           //You code starts here
9
10
11        $display("a = %0d", a);
12      end
13    end
14
15  endmodule
```

Upload 1 supported file: drawing or image. Max 10 MB.

Add file

Answer #1: Best sol. => will display 1000 randomized values within the required range.

```
module tb_try;
  integer a;
  initial begin
    repeat (1000) begin
        a = 8 + 2 * (($random & 31) % 7); // masking with 31 to force positive values only
        $display("a = %0d", a);
    end
  end
endmodule
```

For credibility, I added some of your colleagues' valid answers for Q8 for inspiration. <u>So some of the coming answers are by students.</u>

Answer #2:

```
module tb_try;
  integer a, x; // x is a temp variable
  initial begin
    repeat (1000) begin
        x = (($random&31) % 15) + 7;
        if( x % 2 == 0) begin
            a = x;
            $display("a = %0d", a);
        end
    end
  end
endmodule
```

Answer #3:

```
module tb_try ();
  integer a, temp;

  initial begin
    repeat (1000) begin
      temp = $random;
      if (temp > 7 && temp < 21 && temp[0] == 0) begin
        a = temp;
        $display("a = %0d", a);
      end
    end
  end
endmodule
```

Answer #4:

```
module tb_try;
  integer a;
  initial begin
    repeat (1000) begin
      // Mask to force even positive numbers from 0 to 30
      temp = $random & 32'h0000001E;

      // Check if 'temp' is less than 21
      if (temp < 21 && temp > 7) begin
        a = temp;
        $display("a = %0d", a);
      end
    end
  end
endmodule
```

Answer #5:

```
module tb_try;
  integer a;
  initial begin
    repeat (1000) begin
      a = ((($random % 7) + 7) % 7) * 2 + 8);
      $display("a = %0d", a);
    end
  end
endmodule
```

Common Mistakes: Conditional display.
"a" is given all random values, and the display is conditioned to show the required range of values only.

```
module tb_try ();
integer a;
initial begin
    repeat(1000) begin
        a=$random;
        if(a=>7&&a<=21&&a[0]==0) begin
            $display("a=%0d",a);
        end
    end
end

endmodule
```

Incorrect Answers

Q9) *

```
module tb;

  initial begin
    integer a, res;

    a = 2;

    $display("Before calling fn: a=%0d res=%0d", a, res);

    res = fn(a);

    $display("After calling fn: a=%0d res=%0d", a, res);

    $finish;
  end

  function int fn(int a);
    a = a + 5;
    return a * 10;
  endfunction

endmodule
```

- ○ Before calling fn: a = 2, res = 0. After calling fn: a = 7, res = 70
- ○ Before calling fn: a = 2, res = 0. After calling fn: a = 2, res = 70
- ○ Before calling fn: a = 2, res = x. After calling fn: a = 2, res = 70
- ○ Before calling fn: a = 2, res = x. After calling fn: a = 7, res = 70

Answer:

Before calling fn: a = 2, res = x. After calling fn: a = 2, res = 70

```
VSIM 3> run -all
# c = 1000
# (a === b) equals 0
# (A === b) equals 1
# (a == A) equals x
```

```
VSIM 8> run -all
# Before calling fn: a=2 res=x
# After calling fn: a=2 res=70
```